Project Name: Budgetron 3000

Team/Group: Meet n em

Version: 2.0 (final)

1 Document Version History

Date	Version	Author	Comments
2026-10-14	1.0		Initial design document for the PocketBudget prototype
2026-10-16	1.1		Added AI Budget Coach (separate from MVP) and its guardrails
2026-10-19	1.2		Added C4 context, container and component diagrams, mockup screens and the runnable app demo
2026-10-26	2.0		Final version: renamed to Budgetron 3000; final C4 diagrams at all four levels; demo updated from the main branch; break tests re-run; AI chat, guardrails and supporting analysis updated to match the built app

Table of Contents

- 1 Document Version History
- 2 Overview
  - 2.1 Project Description
  - 2.2 Final Deliverable
- 3 Workflow Diagram
- 4 Architecture and Process flow diagram
  - 4.1 System Context diagram
  - 4.2 Container diagram
  - 4.3 Component diagram
  - 4.4 Code diagrams

- 4.5 Mockup Screens
- 4.6 App Demo
- 4.7 Demo Break Tests

2 Overview

2.1 Project Description

Many students and first-job earners manage money across a paycheck, a few subscriptions and a savings goal with no single place to see it all. Budgetron 3000 is a budgeting website that shows income, spending, savings goals, accounts and upcoming payments together, so a person always knows what is left to spend.

The demo uses a sample profile earning $3,000 a month. Users log purchases and planned payments on a calendar, edit or delete them on Transactions, and see where money goes in an interactive pie chart. Savings goals follow savings balances and estimate how much to save each month, and Accounts lets users plan allocations and simulate transfers between pretend accounts.

An optional AI assistant, Ask Budgetron, answers questions about the app and the user's budget through a local server and Google Gemini; it is separate from the MVP. The app is plain HTML, CSS and JavaScript with a small Node.js server. There are no real bank connections and no real money moves.

2.2 Final Deliverable

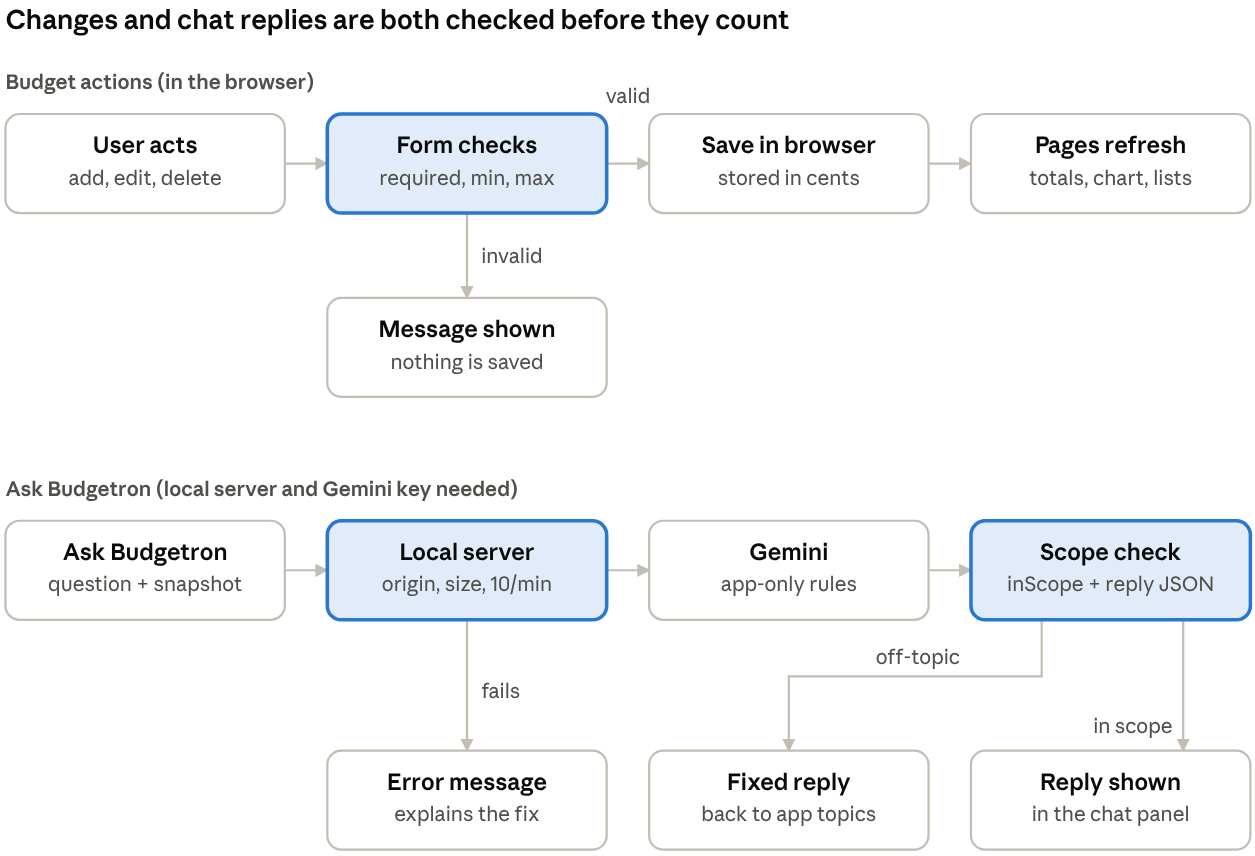
- A short report covering the problem and the value of the solution
  - The core features and how they work: Home summary, savings goals, accounts and allocations, simulated transfers, payment calendar, transactions with edit and delete, and the spending pie chart
  - Ask Budgetron, the optional AI chat, and its guardrails
- The C4 diagrams at all four levels (context, container, component and code) in section 4
- A test plan: 16 automated checks (node --test), the demo test cases, and 56 break tests with pass/fail results (section 4.7)

Requirements and MVP status, business case, competitor research, functional design, the AI feature and its guardrails, screen design and the test plan are in `Supporting analysis`.

mockup and a working model of the app are also deliverables

Both are in section 4: the mockup in 4.5 Mockup Screens and the runnable demo in 4.6 App Demo.

3 Workflow Diagram



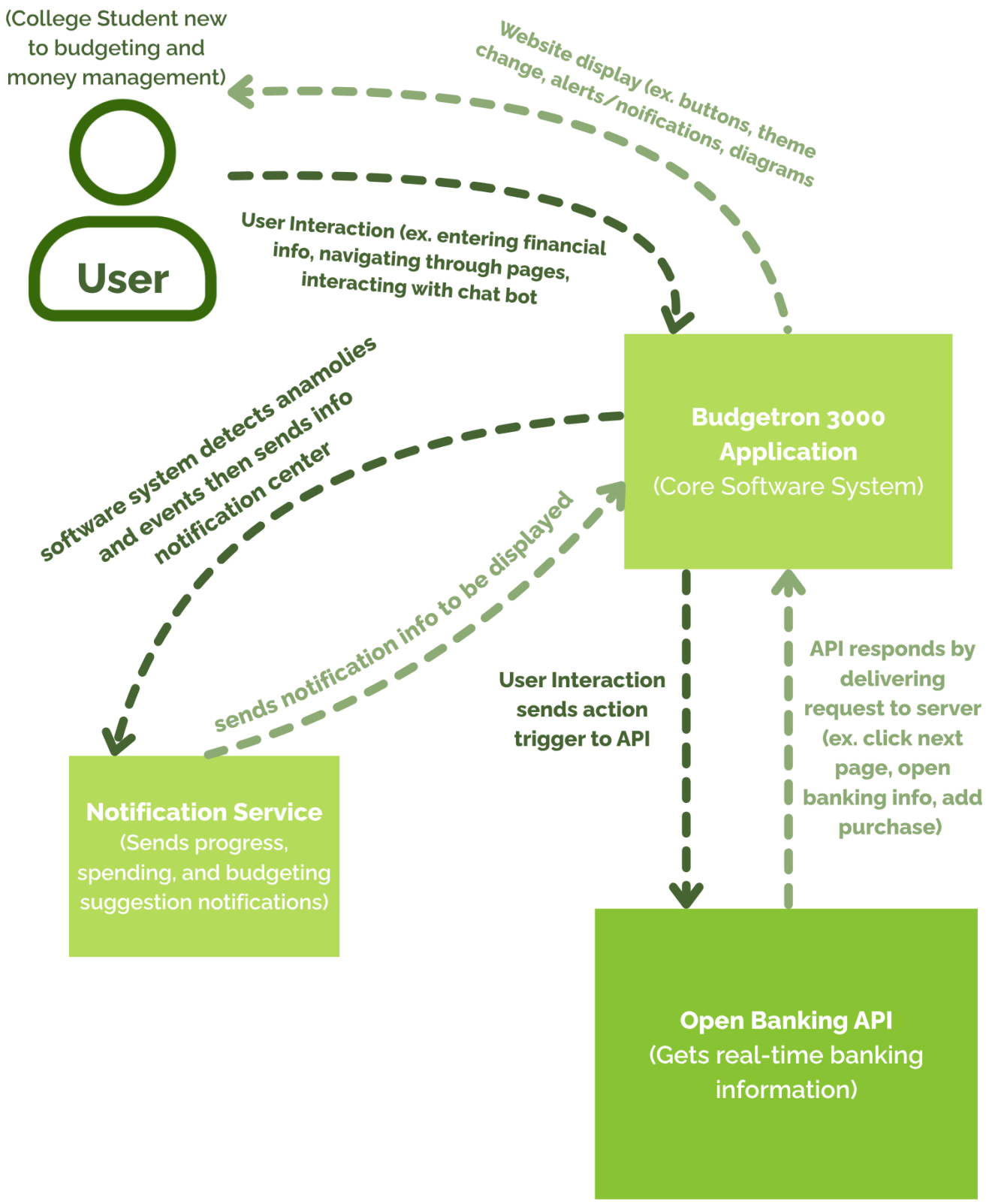
Budgetron 3000 workflow · budget actions and Ask Budgetron

Every budget change passes the form checks before it is saved in the browser, and every page then refreshes from the same data. Chat questions pass the server checks, and Gemini's answer passes a scope check before the user sees it.

4 Architecture and Process flow diagram

4.1 System Context diagram

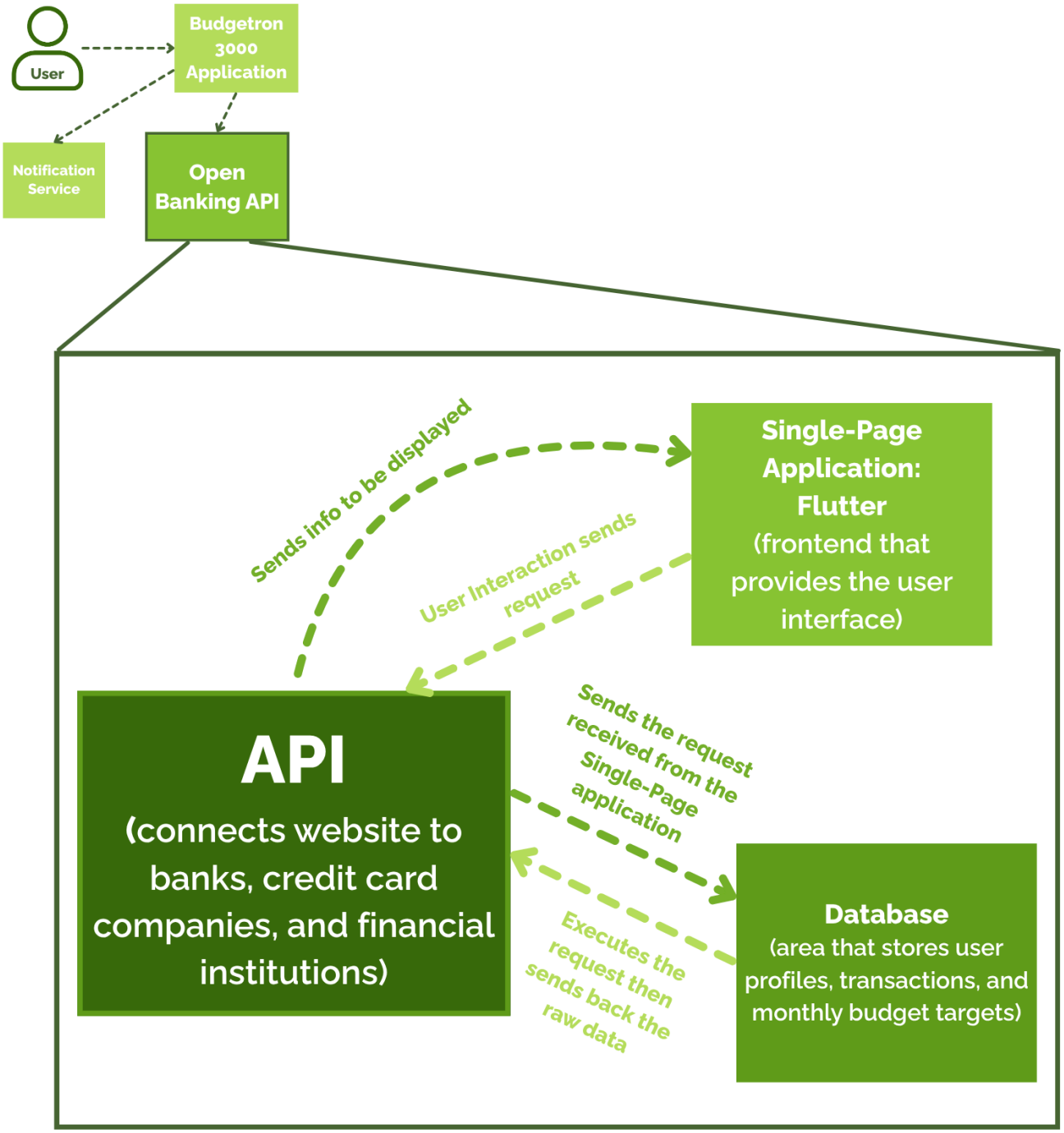
Budgetron 3000 serves one user, a college student new to budgeting and money management. The user enters financial information, moves between pages and chats with the bot, and the app displays the results: buttons, themes, alerts and diagrams. The app sends action triggers to an Open Banking API, which returns real-time banking information. When it detects anomalies or events, it notifies a Notification Service, which sends progress, spending and budgeting-suggestion alerts.


(College Student new to budgeting and money management)
User
Website display (ex. buttons, theme change, alerts/noifications, diagrams
User Interaction (ex. entering financial info, navigating through pages, interacting with chat bot
Budgetron 3000 Application
(Core Software System)
software system detects anamolies and events then sends info notification center
sends notification info to be displayed
User Interaction sends action trigger to API
API responds by delivering request to server (ex. click next page, open banking info, add purchase)
Notification Service
(Sends progress, spending, and budgeting suggestion notifications)
Open Banking API
(Gets real-time banking information)

In the class demo, the bank and notification services are simulated: balances are sample data and alerts appear inside the app.

4.2 Container diagram

The system has three containers. A single-page application gives the user interface and sends user requests to the API. The API connects the website to banks, credit card companies and financial institutions; it sends requests to the database, receives raw data back and returns what the interface should display. The database stores user profiles, transactions and monthly budget targets.

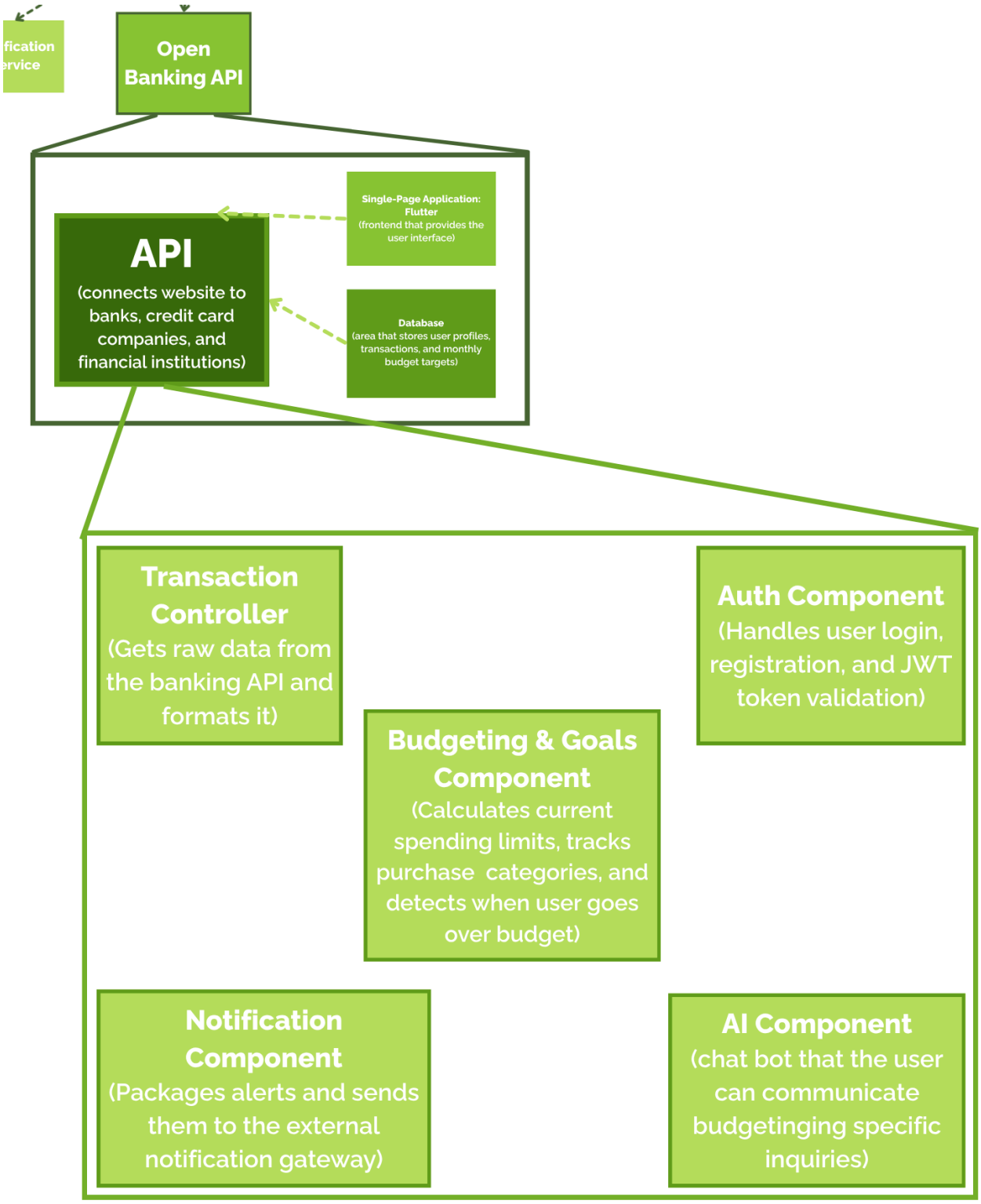

User
Budgetron 3000 Application
Notification Service
Open Banking API
Sends info to be displayed
Single-Page Application: Flutter (frontend that provides the user interface)
User Interaction sends request
API
(connects website to banks, credit card companies, and financial institutions)
Sends the request received from the Single-Page application
Executes the request then sends back the raw data
Database
(area that stores user profiles, transactions, and monthly budget targets)

The design names Flutter for the interface. The class demo builds it in plain HTML, CSS and JavaScript and keeps data in browser storage instead of a database; its Node.js server handles AI chat only.

4.3 Component diagram

The API is split into five components:

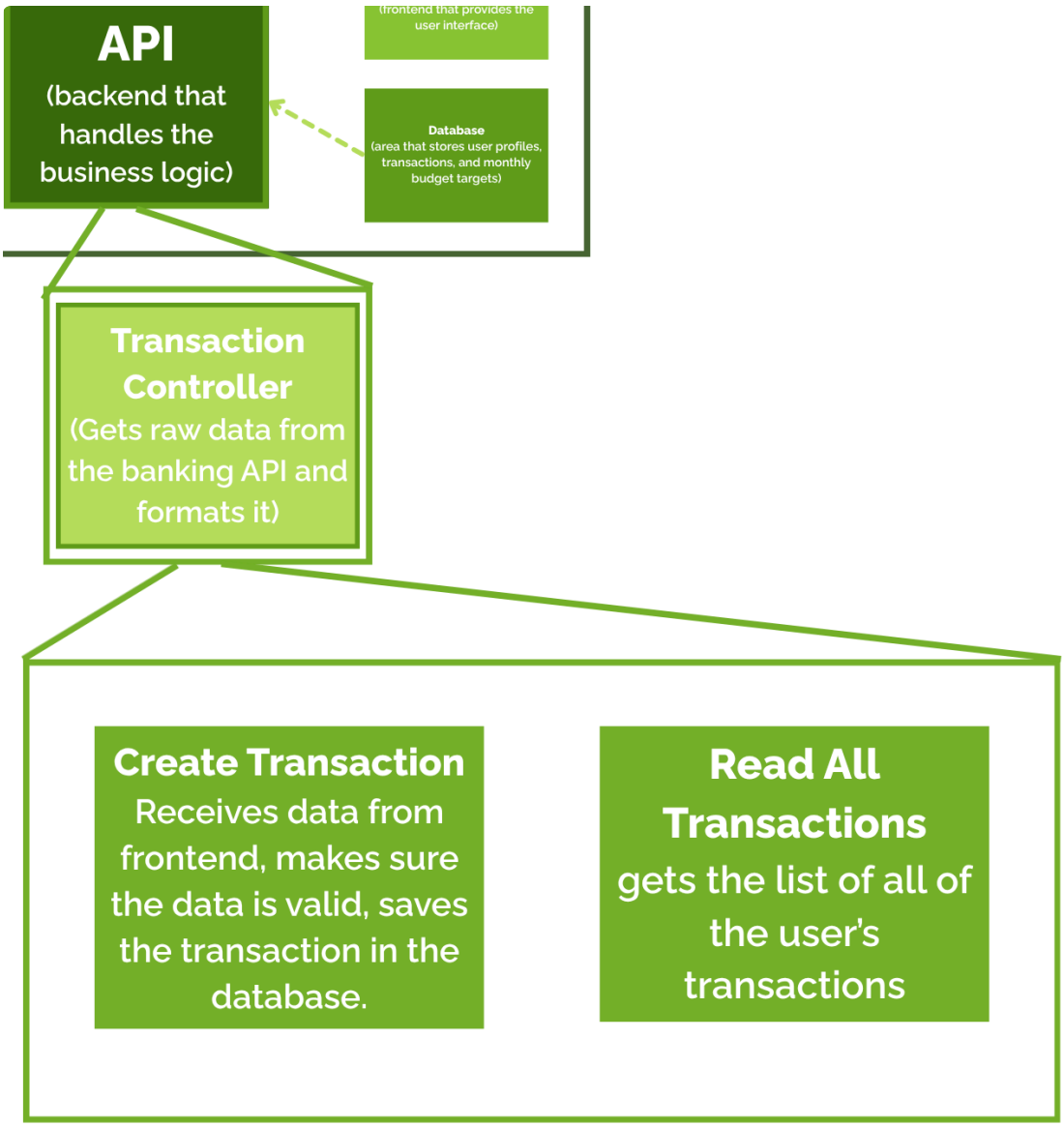
- **Transaction Controller:** gets raw data from the banking API and formats it.
- **Budgeting & Goals Component:** calculates current spending limits, tracks purchase categories and detects when the user goes over budget.
- **Auth Component:** handles login, registration and JWT token validation.
- **Notification Component:** packages alerts and sends them to the external notification gateway.
- **AI Component:** the chatbot the user asks budgeting questions, built as Ask Budgetron (separate from the MVP).


ification
ervice
Open Banking API
API
(connects website to banks, credit card companies, and financial institutions)
Single-Page Application: Flutter
(frontend that provides the user interface)
Database
(area that stores user profiles, transactions, and monthly budget targets)
Transaction Controller
(Gets raw data from the banking API and formats it)
Auth Component
(Handles user login, registration, and JWT token validation)
Budgeting & Goals Component
(Calculates current spending limits, tracks purchase categories, and detects when user goes over budget)
Notification Component
(Packages alerts and sends them to the external notification gateway)
AI Component
(chat bot that the user can communicate budgetinging specific inquiries)

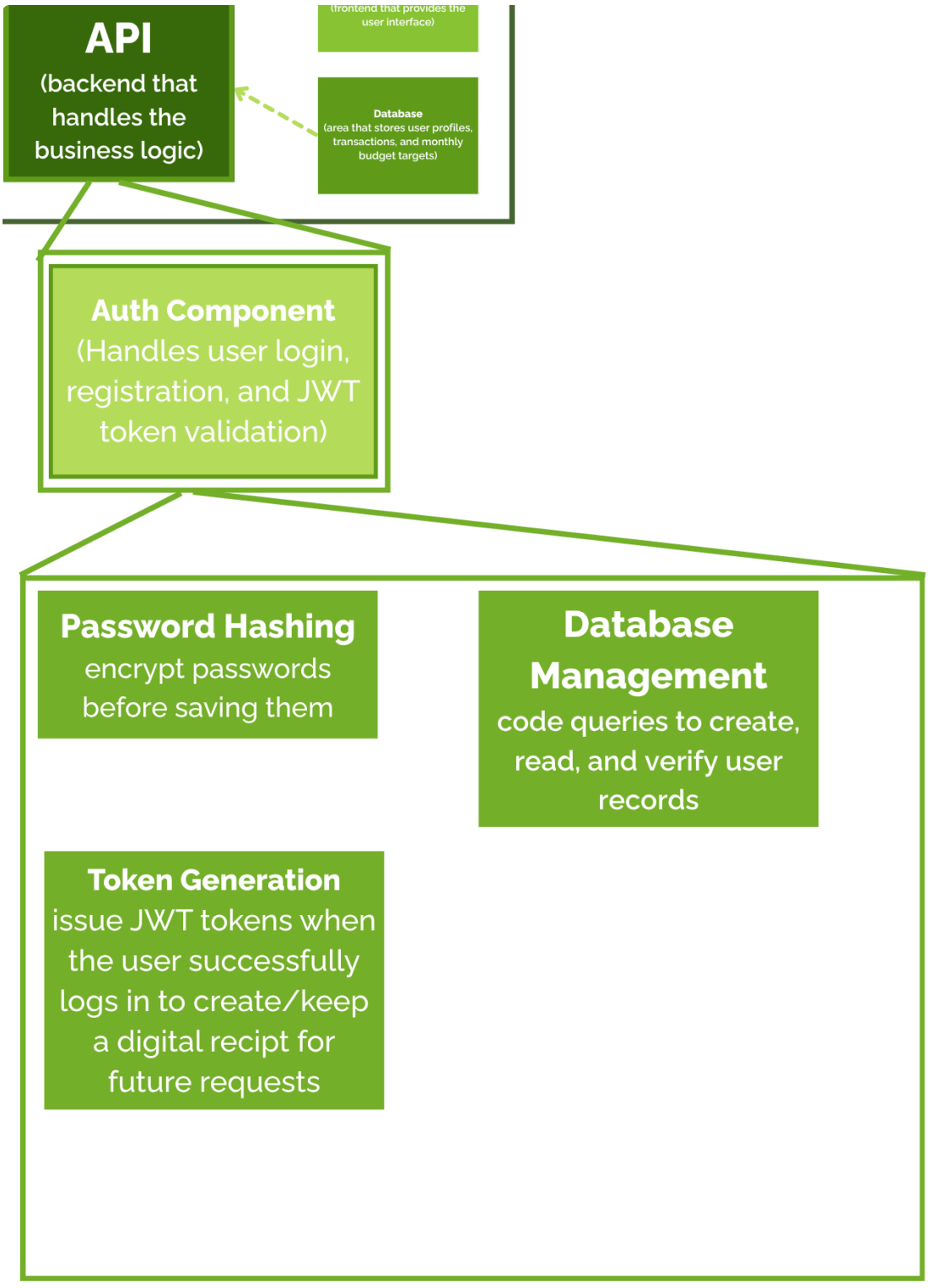
4.4 Code diagrams

Four components are broken down to code level; the Notification Component has no code diagram.

Transaction Controller: Create Transaction receives data from the frontend, checks it is valid and saves it in the database. Read All Transactions returns the list of all the user's transactions.


API
(backend that handles the business logic)
(frontend that provides the user interface)
Database
(area that stores user profiles, transactions, and monthly budget targets)
Transaction Controller
(Gets raw data from the banking API and formats it)
Create Transaction
Receives data from frontend, makes sure the data is valid, saves the transaction in the database.
Read All Transactions
gets the list of all of the user's transactions

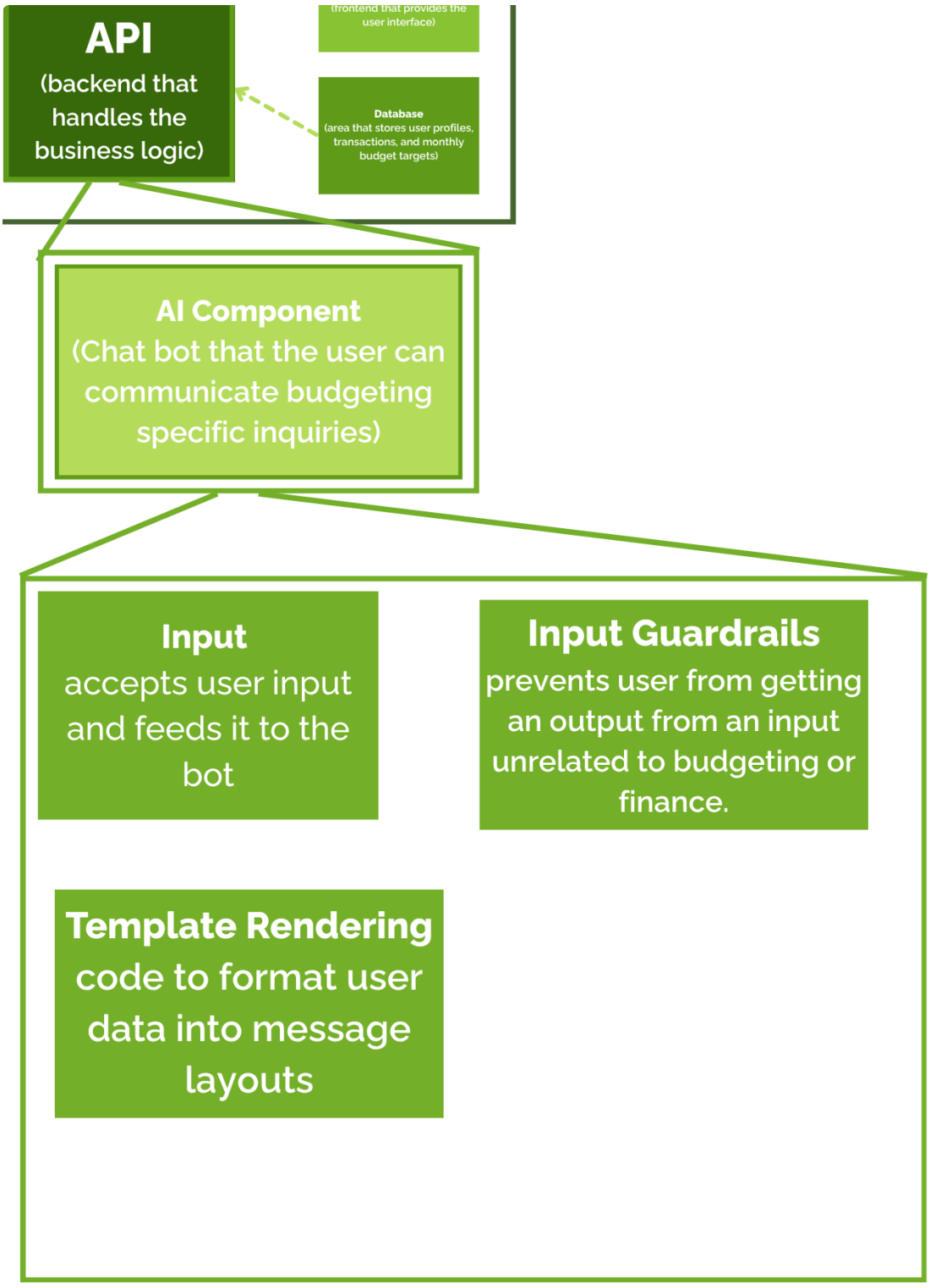
Auth Component: Password Hashing encrypts passwords before they are saved. Database Management runs the queries that create, read and verify user records. Token Generation issues JWT tokens when the user logs in, as a digital receipt for later requests.


API
(backend that handles the business logic)
(frontend that provides the user interface)
Database
(area that stores user profiles, transactions, and monthly budget targets)
Auth Component
(Handles user login, registration, and JWT token validation)
Password Hashing
encrypt passwords before saving them
Database Management
code queries to create, read, and verify user records
Token Generation
issue JWT tokens when the user successfully logs in to create/keep a digital recipt for future requests

Budgeting & Goals Component: Monthly Spending Calculator gathers spending from the API and the user's own entries. Compute Remaining Balances sums expenses by category, calculates the remaining budget and triggers when the user approaches or passes a threshold. Goal Progress Monitor calculates progress percentage and the required savings rate. Keep Track of Goals holds the list of goals, their estimated completion and whether each is complete.


API
(backend that handles the business logic)
(frontend that provides the user interface)
Database
(area that stores user profiles, transactions, and monthly budget targets)
Budgeting & Goals Component
(Calculates current spending limits, tracks purchase categories, and detects when user goes over budget)
Monthly Spending Calculator
gathers the user's spending data from information gathered from the API and from the user's spendign input
Goal Progress Monitor
calculates progress percentage, required savings rate, and allocates funds to a goal
Compute Remaining Balances
sum all expenses grouped by category for the goals's time period, calculate the remaining budget, and trigger when user is approaching/has passed the balance threshold
Keep Track of Goals
creates a mutible list that keeps track of the user's goals, the estimated completion time, and if the goal has been completed

AI Component: Input accepts the user's message and feeds it to the bot. Input Guardrails stop the user getting an answer to anything unrelated to budgeting or finance. Template Rendering formats the user's data into message layouts.



Where each code-level part lives in the demo:

Code-level part	In the demo
Create Transaction	Calendar add form and the Transactions Edit dialog: required fields, min and max amounts, then save
Read All Transactions	Transactions page list, newest first, with Edit and Delete
Password Hashing, Token Generation, Database Management	Not built: the demo has no sign-in and stores data in the browser
Monthly Spending Calculator, Compute Remaining Balances	`spendingTotals()` and `currentMonthPurchases()` in state.js; totals on Home and Transactions
Goal Progress Monitor, Keep Track of Goals	`goalSavings()` in state.js and the Goals page progress ring and monthly estimate
Input	Ask Budgetron chat panel (chat.js)
Input Guardrails	Server request checks (server.mjs) and the scope rules and inScope check (chat-policy.mjs)
Template Rendering	Budget snapshot builder and safe reply formatting (chat.js)

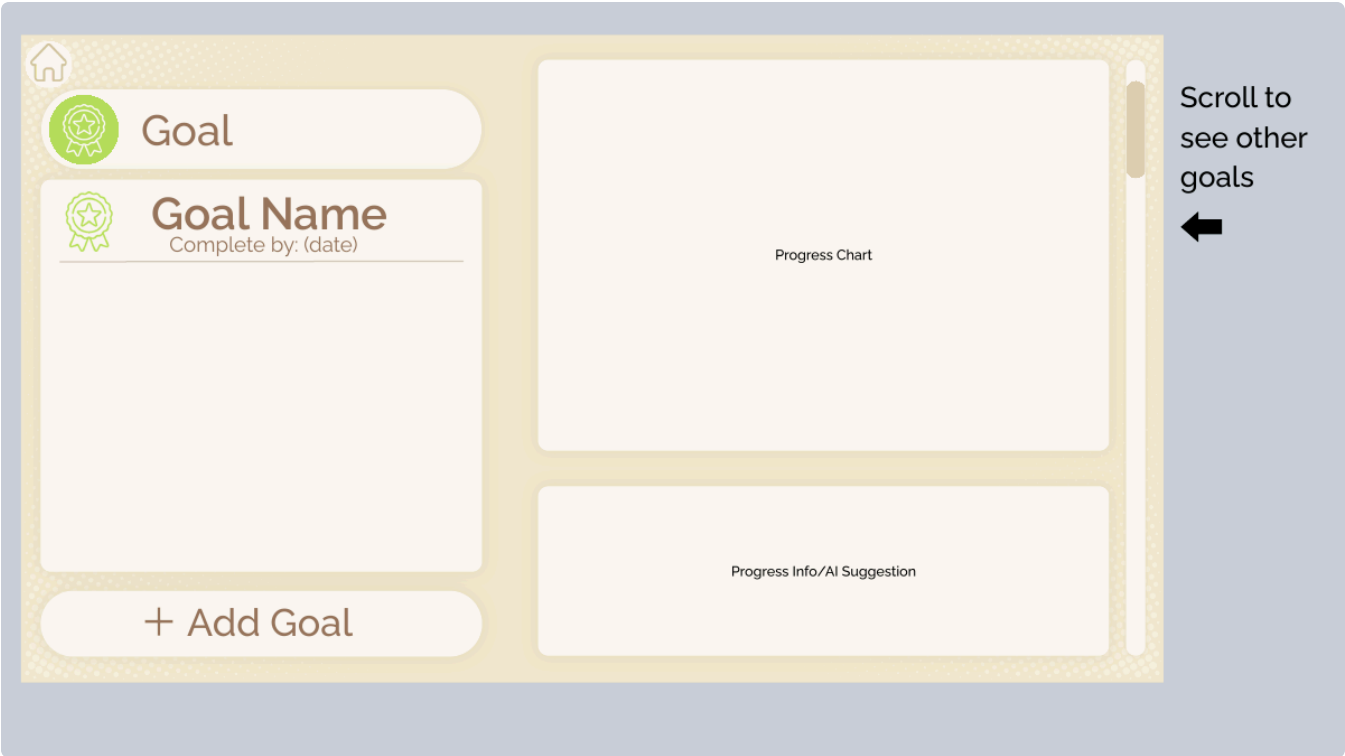
4.5 Mockup Screens

The CS 100 App Mock-Up sets the look of the app: a cream background, brown text, green round icons and a rounded side menu. It has eight screens across five pages: Calendar, Goal, Card Info, Pie Chart (one view per category) and Spending Info.

Calendar: month view with a menu, then a Today timeline of deposits, paydays and bills due.



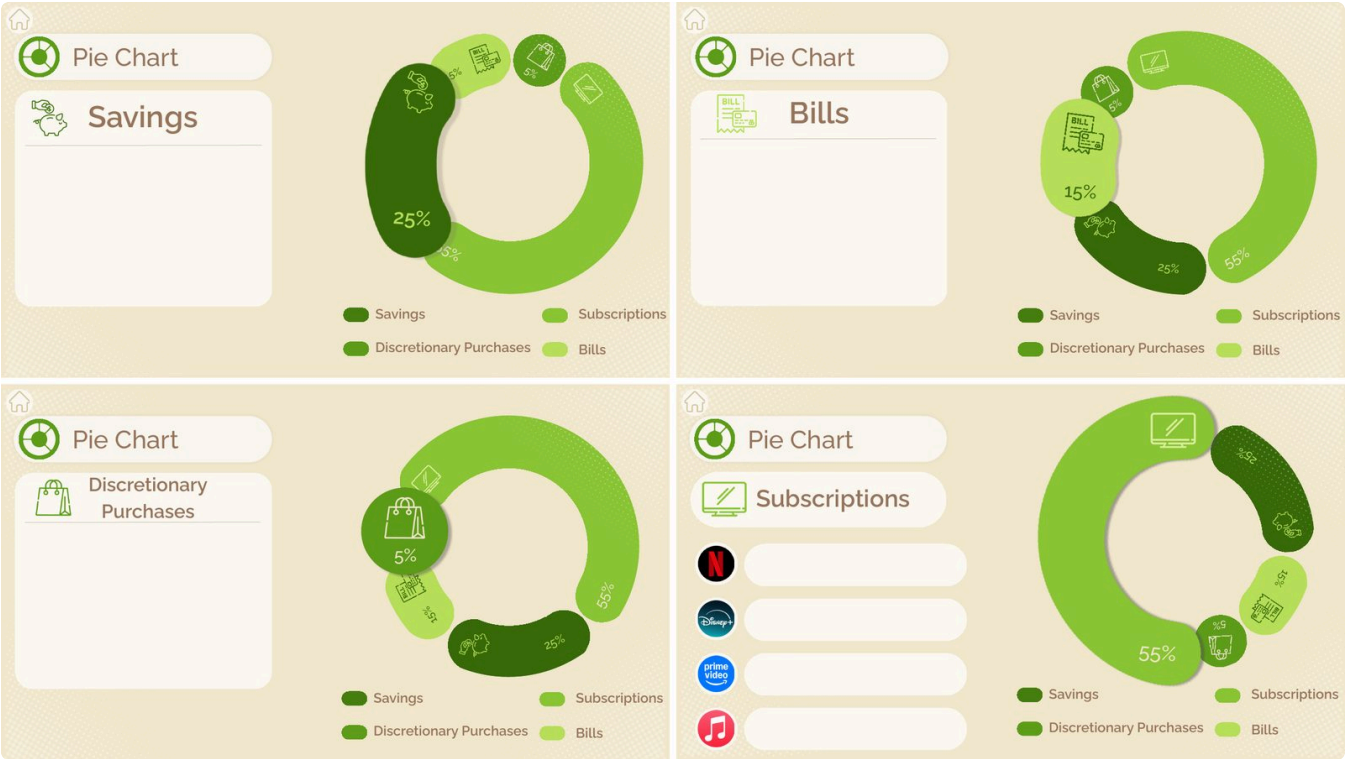
Goal: goal name and completion date, a progress chart, a progress info / AI suggestion panel and Add Goal.



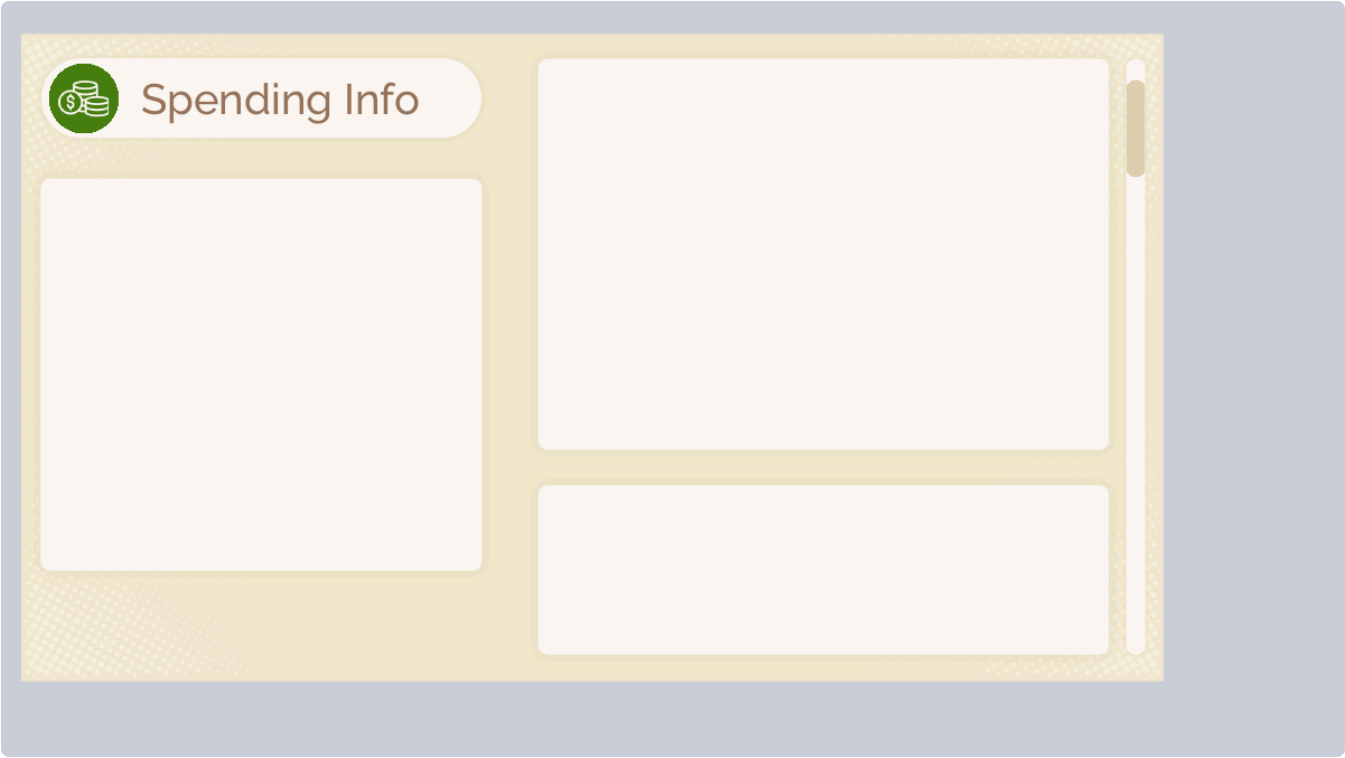
Card Info: stacked bank cards; selecting one shows its details, with Add Card.



Pie Chart: spending split into Savings, Bills, Discretionary Purchases and Subscriptions; selecting a slice raises it and opens that category.



Spending Info: panels for monthly spending details.



4.6 App Demo

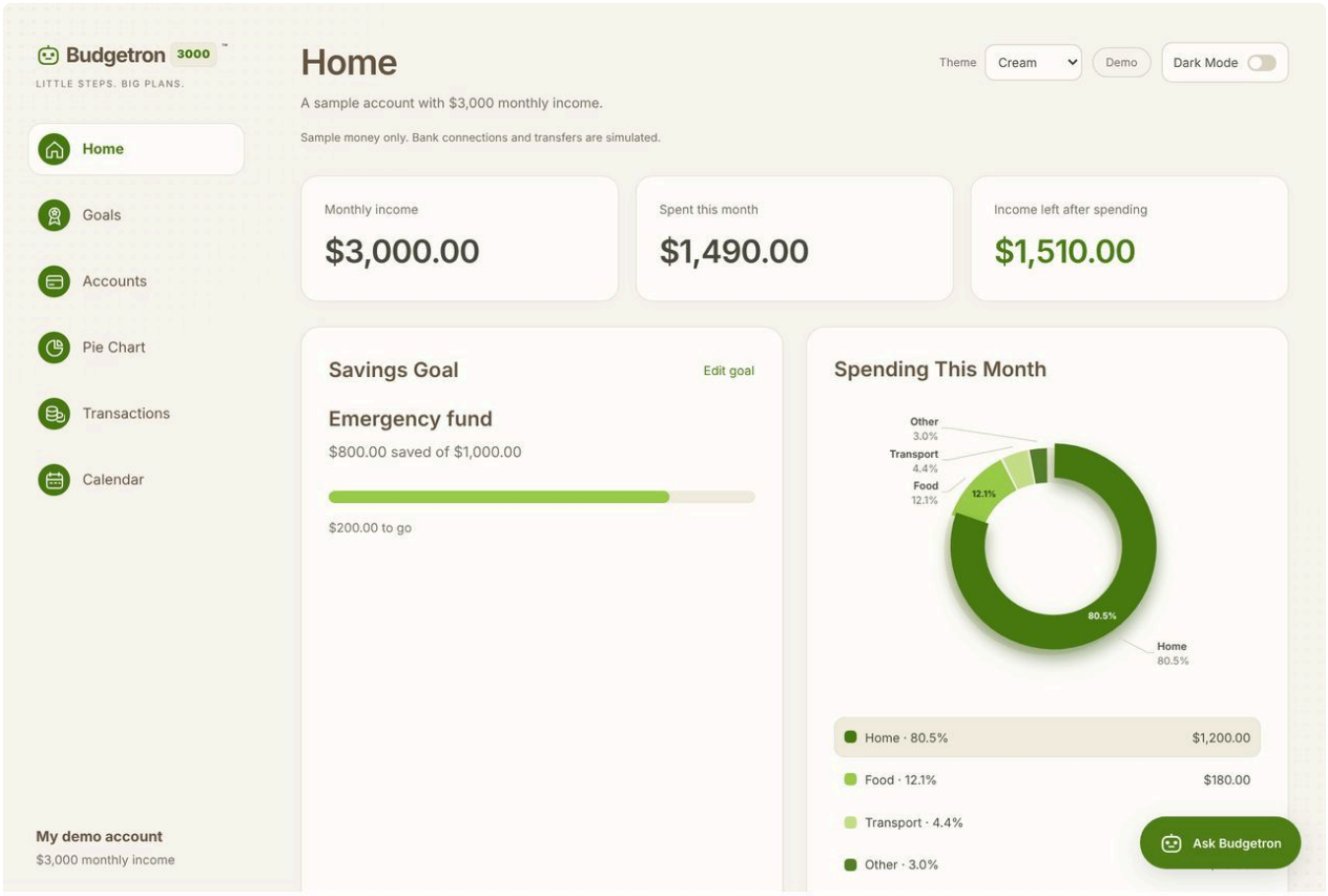
The working demo is Budgetron 3000 from the main branch of the team's GitHub repository. Open the Budgetron 3000 demo to run through it in a browser, or double-click index.html in the repository; no install or account is needed.

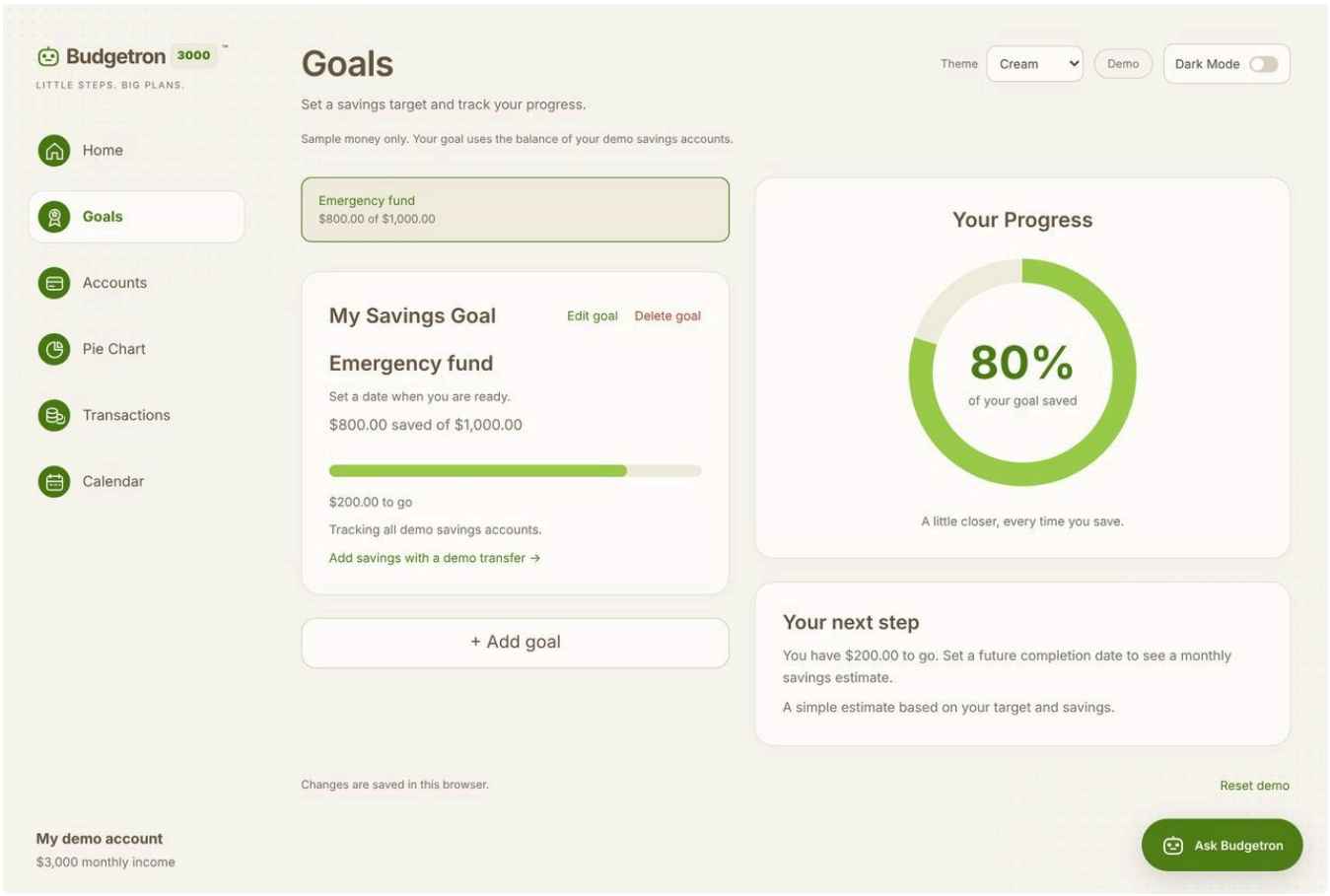
Ask Budgetron needs the local server: install Node.js 22 or newer, copy .env.example to .env, add a Gemini API key, then run Start Budgetron.bat and use the website it opens. Without the server, the chat explains how to start it.

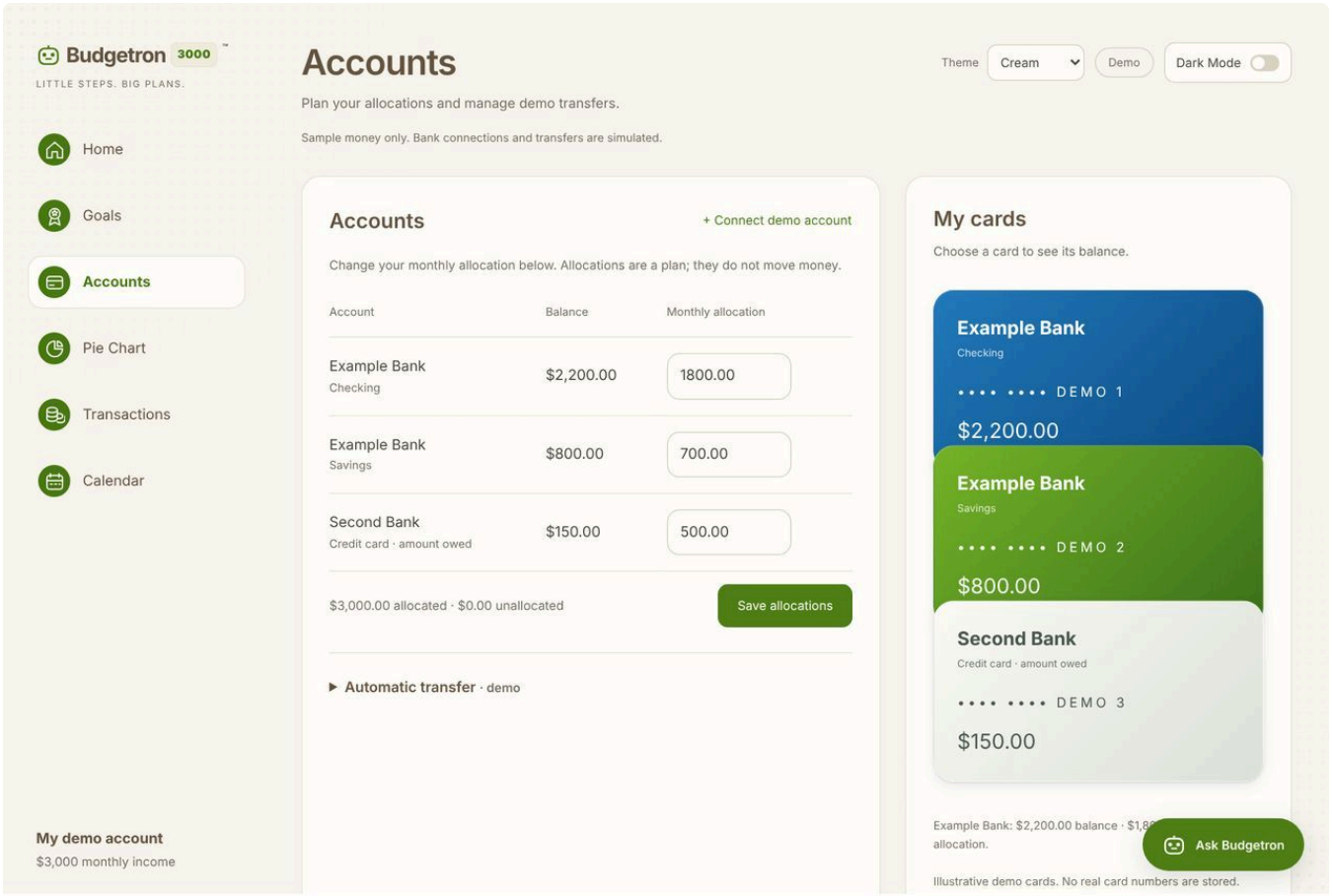
Changes save in the browser, and Reset demo restores the sample. Every page has a Theme menu (Cream, Sage, Ocean, Rose) and a Dark Mode switch.

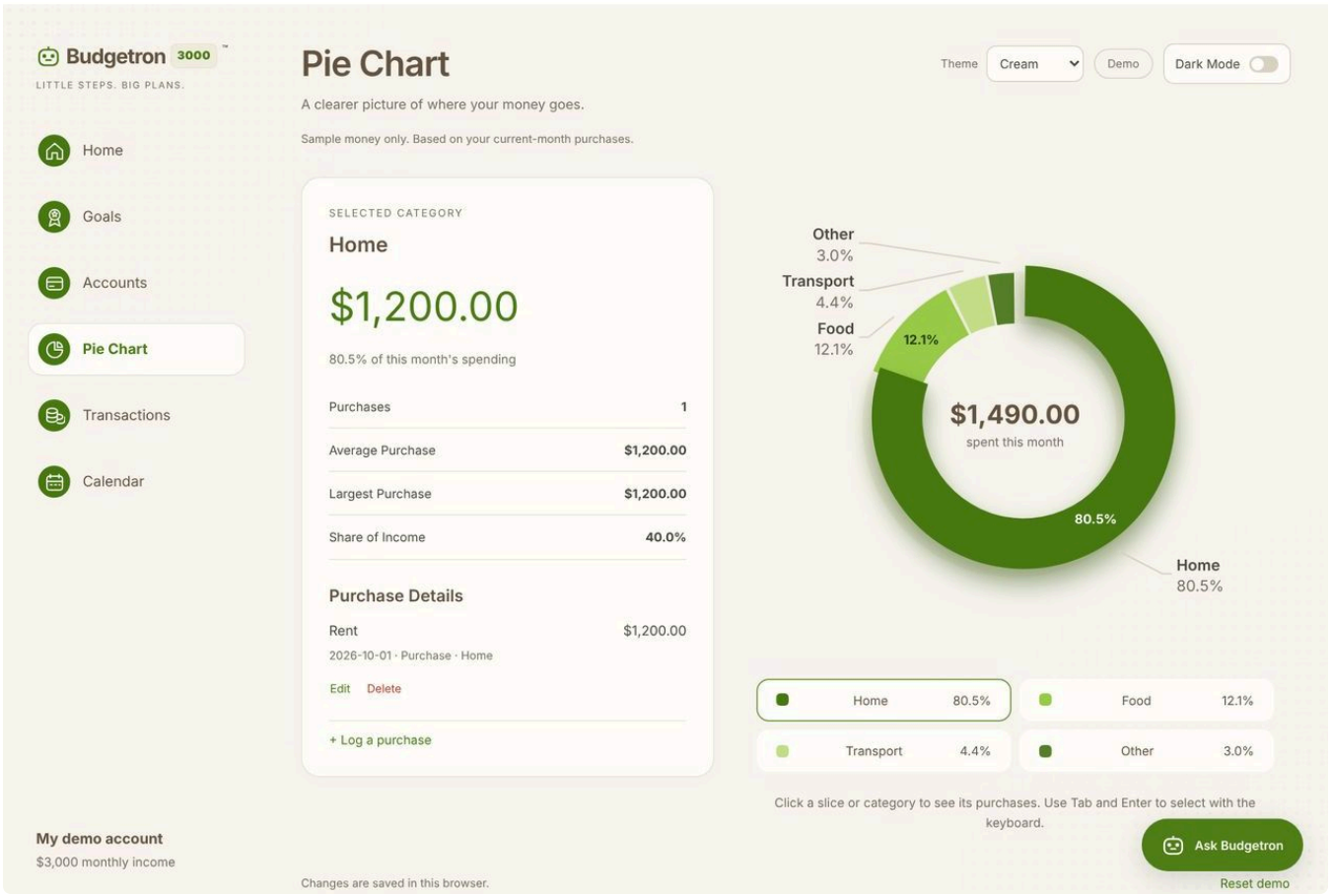
Page	What you can do
Home	See income, spending and money left; the selected savings goal; a spending donut with clickable slices; the three most recent purchases with Edit and Delete
Goals	Add, select, edit and delete goals (delete asks first and never changes balances); set a target, optional date and which savings account to track; see a progress ring and a monthly savings estimate

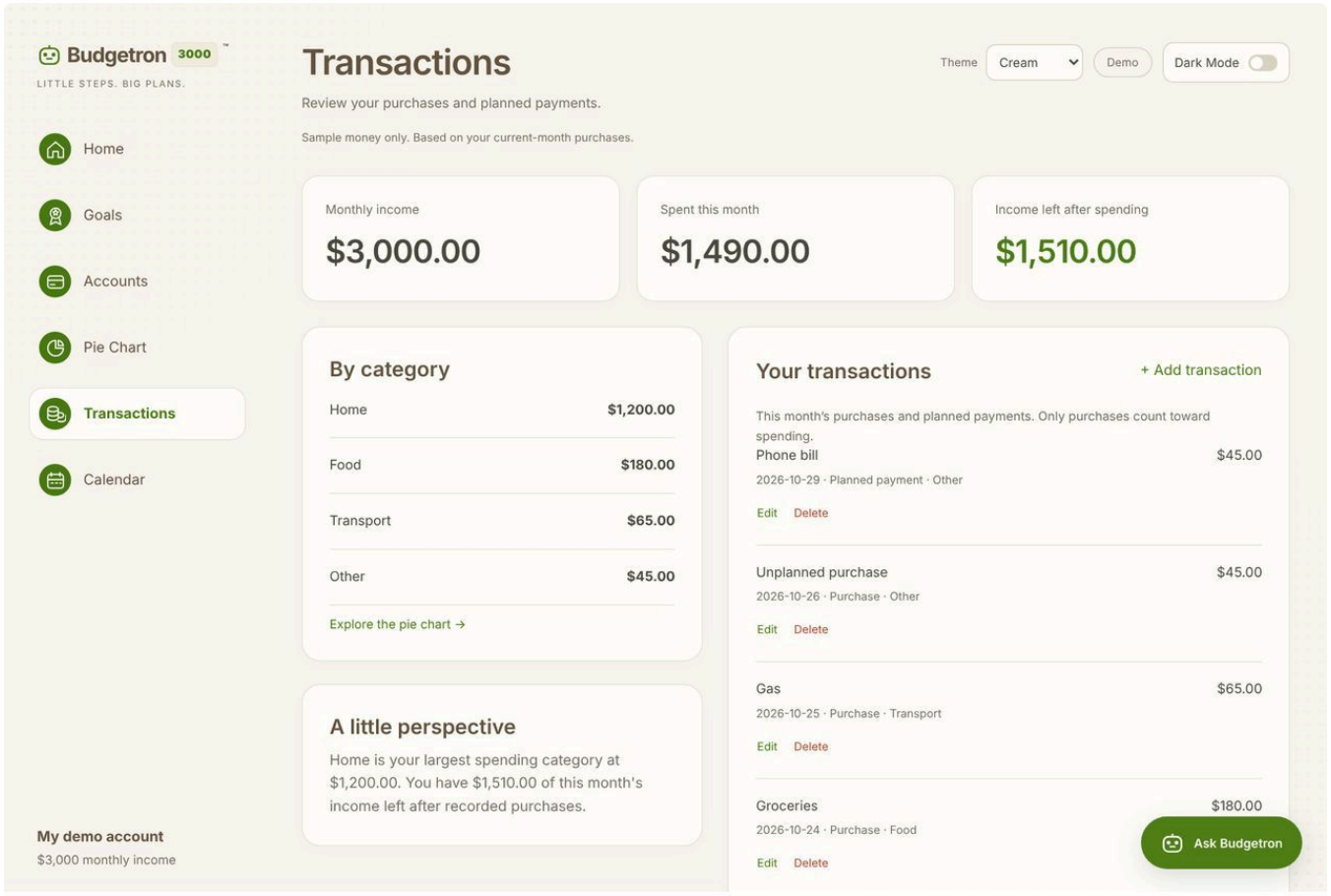
Page	What you can do
Accounts	Edit monthly allocations (total capped at $3,000), connect demo accounts, pick a wallet card, save and simulate a transfer rule
Pie Chart	Select a category to see its total, share of spending and income, purchase count, average and largest purchase, and dated purchases
Transactions	See this month's purchases and planned payments, category totals and a spending summary; Edit or Delete any entry
Calendar	Browse months, jump to Today or All events, add planned payments or unexpected purchases, see the next demo transfer
Ask Budgetron	Ask with suggested questions, choose whether to include the sample budget, start a New Chat, drag the button or show it only on Home

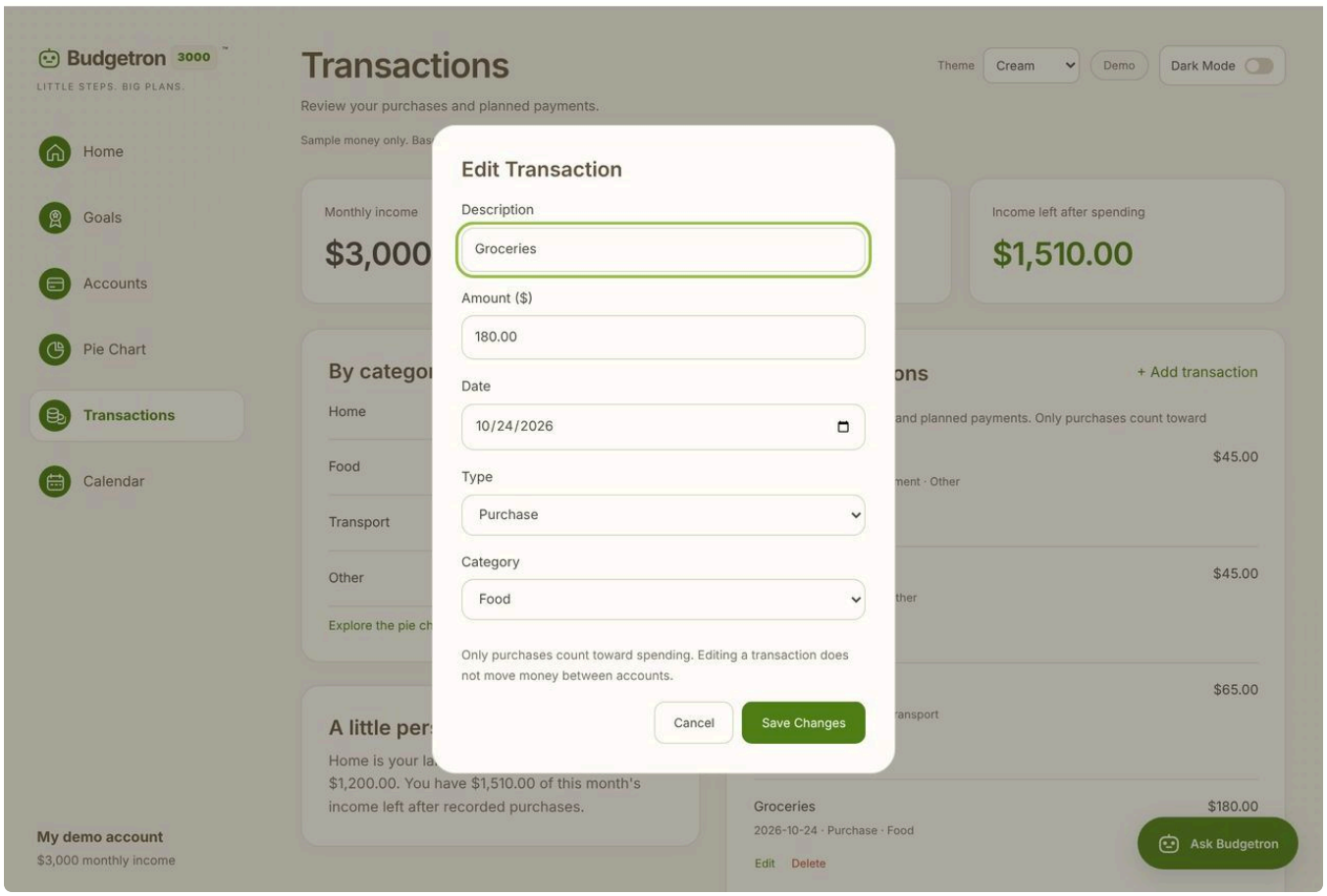

Budgetron 3000
LITTLE STEPS. BIG PLANS.
Home
Goals
Accounts
Pie Chart
Transactions
Calendar
My demo account
$3,000 monthly income
Home
A sample account with $3,000 monthly income.
Sample money only. Bank connections and transfers are simulated.
Theme
Cream
Demo
Dark Mode
Monthly income
$3,000.00
Spent this month
$1,490.00
Income left after spending
$1,510.00
Savings Goal
Edit goal
Emergency fund
$800.00 saved of $1,000.00
$200.00 to go
Spending This Month
Other
3.0%
Transport
4.4%
Food
12.1%
12.1%
80.5%
Home
80.5%
Home · 80.5%
$1,200.00
Food · 12.1%
$180.00
Transport · 4.4%
Other · 3.0%
Ask Budgetron

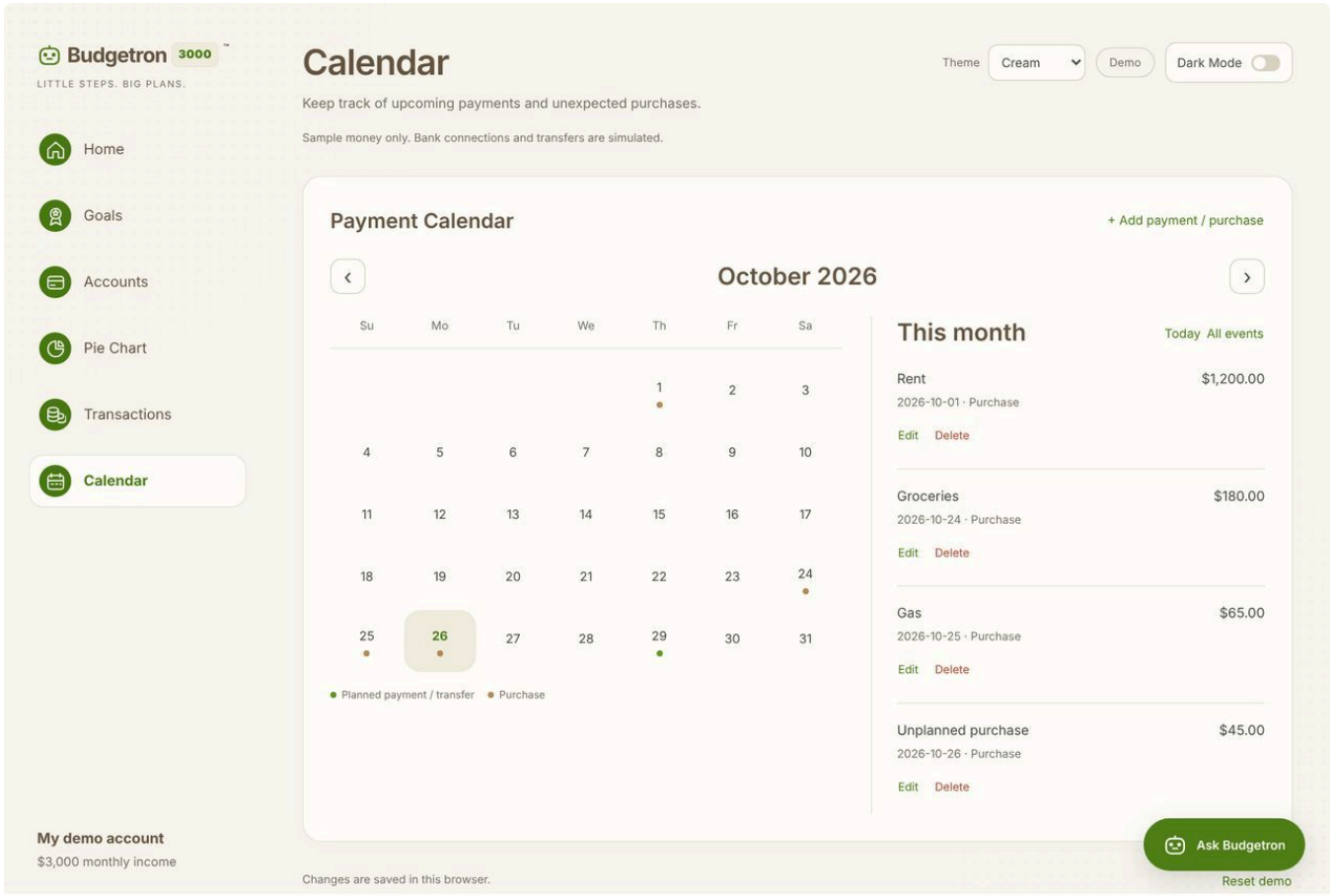

Budgetron 3000
LITTLE STEPS. BIG PLANS.
Home
Goals
Accounts
Pie Chart
Transactions
Calendar
My demo account
$3,000 monthly income
Goals
Set a savings target and track your progress.
Sample money only. Your goal uses the balance of your demo savings accounts.
Theme
Cream
Demo
Dark Mode
Emergency fund
$800.00 of $1,000.00
My Savings Goal
Edit goal
Delete goal
Emergency fund
Set a date when you are ready.
$800.00 saved of $1,000.00
$200.00 to go
Tracking all demo savings accounts.
Add savings with a demo transfer →
+ Add goal
Your Progress
80%
of your goal saved
A little closer, every time you save.
Your next step
You have $200.00 to go. Set a future completion date to see a monthly savings estimate.
A simple estimate based on your target and savings.
Changes are saved in this browser.
Reset demo
Ask Budgetron

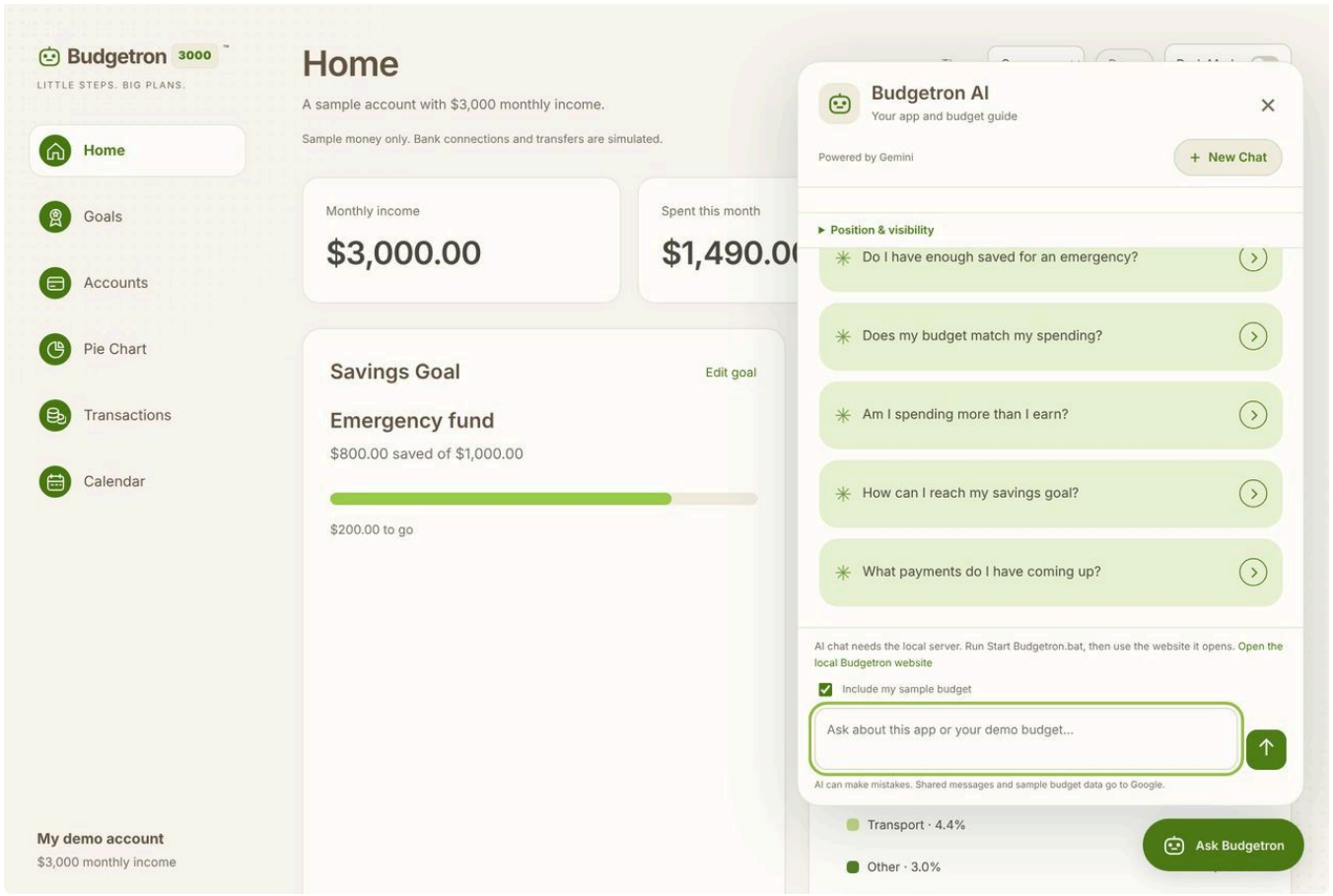

Budgetron 3000
LITTLE STEPS. BIG PLANS.
Home
Goals
Accounts
Pie Chart
Transactions
Calendar
My demo account
$3,000 monthly income
Accounts
Plan your allocations and manage demo transfers.
Sample money only. Bank connections and transfers are simulated.
Theme
Cream
Demo
Dark Mode
Accounts
+ Connect demo account
Change your monthly allocation below. Allocations are a plan; they do not move money.
Account
Balance
Monthly allocation
Example Bank
Checking
$2,200.00
1800.00
Example Bank
Savings
$800.00
700.00
Second Bank
Credit card · amount owed
$150.00
500.00
$3,000.00 allocated · $0.00 unallocated
Save allocations
▶ Automatic transfer · demo
My cards
Choose a card to see its balance.
Example Bank
Checking
•••• •••• DEMO 1
$2,200.00
Example Bank
Savings
•••• •••• DEMO 2
$800.00
Second Bank
Credit card · amount owed
•••• •••• DEMO 3
$150.00
Example Bank: $2,200.00 balance · $1,8
allocation.
Ask Budgetron
Illustrative demo cards. No real card numbers are stored.


Budgetron 3000
LITTLE STEPS. BIG PLANS.
Home
Goals
Accounts
Pie Chart
Transactions
Calendar
My demo account
$3,000 monthly income
Pie Chart
A clearer picture of where your money goes.
Sample money only. Based on your current-month purchases.
Theme
Cream
Demo
Dark Mode
SELECTED CATEGORY
Home
$1,200.00
80.5% of this month's spending
Purchases 1
Average Purchase $1,200.00
Largest Purchase $1,200.00
Share of Income 40.0%
Purchase Details
Rent $1,200.00
2026-10-01 · Purchase · Home
Edit Delete
+ Log a purchase
Other
3.0%
Transport
4.4%
Food
12.1%
12.1%
$1,490.00
spent this month
80.5%
Home
80.5%
Home 80.5%
Food 12.1%
Transport 4.4%
Other 3.0%
Click a slice or category to see its purchases. Use Tab and Enter to select with the keyboard.
Ask Budgetron
Changes are saved in this browser.
Reset demo


Budgetron 3000
LITTLE STEPS. BIG PLANS.
Home
Goals
Accounts
Pie Chart
Transactions
Calendar
My demo account
$3,000 monthly income
Transactions
Review your purchases and planned payments.
Sample money only. Based on your current-month purchases.
Theme
Cream
Demo
Dark Mode
Monthly income
$3,000.00
Spent this month
$1,490.00
Income left after spending
$1,510.00
By category
Home $1,200.00
Food $180.00
Transport $65.00
Other $45.00
Explore the pie chart →
A little perspective
Home is your largest spending category at $1,200.00. You have $1,510.00 of this month's income left after recorded purchases.
Your transactions
+ Add transaction
This month's purchases and planned payments. Only purchases count toward spending.
Phone bill $45.00
2026-10-29 · Planned payment · Other
Edit Delete
Unplanned purchase $45.00
2026-10-26 · Purchase · Other
Edit Delete
Gas $65.00
2026-10-25 · Purchase · Transport
Edit Delete
Groceries $180.00
2026-10-24 · Purchase · Food
Edit Delete
Ask Budgetron


Budgetron 3000
LITTLE STEPS. BIG PLANS.
Home
Goals
Accounts
Pie Chart
Transactions
Calendar
My demo account
$3,000 monthly income
Transactions
Review your purchases and planned payments.
Theme
Cream
Demo
Dark Mode
Edit Transaction
Description
Groceries
Amount ($)
180.00
Date
10/24/2026
Type
Purchase
Category
Food
Only purchases count toward spending. Editing a transaction does not move money between accounts.
Cancel
Save Changes
Income left after spending
$1,510.00
+ Add transaction
Groceries $180.00
2026-10-24 · Purchase · Food
Edit Delete
Ask Budgetron


Budgetron 3000
LITTLE STEPS. BIG PLANS.
Home
Goals
Accounts
Pie Chart
Transactions
Calendar
My demo account
$3,000 monthly income
Calendar
Keep track of upcoming payments and unexpected purchases.
Sample money only. Bank connections and transfers are simulated.
Theme
Cream
Demo
Dark Mode
Payment Calendar
+ Add payment / purchase
October 2026
Su Mo Tu We Th Fr Sa
1 2 3
4 5 6 7 8 9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
Planned payment / transfer
Purchase
This month
Today
All events
Rent
$1,200.00
2026-10-01 · Purchase
Edit Delete
Groceries
$180.00
2026-10-24 · Purchase
Edit Delete
Gas
$65.00
2026-10-25 · Purchase
Edit Delete
Unplanned purchase
$45.00
2026-10-26 · Purchase
Edit Delete
Ask Budgetron
Changes are saved in this browser.
Reset demo


Budgetron 3000
LITTLE STEPS. BIG PLANS.
Home
Goals
Accounts
Pie Chart
Transactions
Calendar
My demo account
$3,000 monthly income
Home
A sample account with $3,000 monthly income.
Sample money only. Bank connections and transfers are simulated.
Monthly income
$3,000.00
Spent this month
$1,490.00
Savings Goal
Edit goal
Emergency fund
$800.00 saved of $1,000.00
$200.00 to go
Budgetron AI
Your app and budget guide
Powered by Gemini
+ New Chat
Position & visibility
Do I have enough saved for an emergency?
Does my budget match my spending?
Am I spending more than I earn?
How can I reach my savings goal?
What payments do I have coming up?
AI chat needs the local server. Run Start Budgetron.bat, then use the website it opens. Open the local Budgetron website
Include my sample budget
Ask about this app or your demo budget...
AI can make mistakes. Shared messages and sample budget data go to Google.
Transport · 4.4%
Other · 3.0%
Ask Budgetron

4.7 Demo Break Tests

53 of 56 break tests passed and 3 failed (B21, B29, B37). They were re-run on 2026-10-26 against the main branch: 43 tests in the browser on the six pages and the chat panel, and 13 sent straight to the AI chat server. The team's own suite (`node --test`) also passes all 16 checks. Failures are recorded here and the app was not changed.

Since the first run, B33 and B34 now pass because editing and deleting purchases and goals were added; B36 to B43 cover the new features. Live Gemini answers were not tested because no API key was used; the scope check itself is covered by the team's chat-policy tests.

Browser tests

ID	Test	Expected	Actual	Pass / Fail
B1	Accounts: allocate $3,000 to Checking while Savings and Credit keep $700 and $500 (total $4,200)	Rejected with a message; nothing saved	Error shown: "Keep total allocations at or below your $3,000 income."; not saved	Pass
B2	Accounts: enter -50 as an allocation and save	Blocked; nothing saved	Browser blocked the form (minimum 0); nothing saved	Pass
B3	Accounts: clear an allocation box and save	Blocked; nothing saved	Blocked as a required field; nothing saved	Pass
B4	Goals: save a goal whose name is only spaces	Rejected; asks for a name	Dialog stayed open asking for a name; goal still "Emergency fund"	Pass
B5	Goals: set the target to $0	Blocked; nothing saved	Blocked (minimum $0.01); nothing saved	Pass
B6	Goals: set the target to -$100	Blocked; nothing saved	Blocked (minimum $0.01); nothing saved	Pass
B7	Goals: name a goal `<img src=x onerror=alert(1)>`	Shown as plain text; no code runs	Shown as plain text; no script ran	Pass

ID	Test	Expected	Actual	Pass / Fail
B8	Goals: set a completion date in the past (2020-01-01)	Rejected or warned that the date has passed	Saved "Complete by 2020-01-01"; tip: "You have $700.00 to go. Set a future completion date to see a monthly savings estimate."	Pass
B9	Goals: set a $500 target when $800 is already saved	Shows 100% and Goal reached, not 160%	Ring shows 100%; "Goal reached!"	Pass
B10	Goals: set a $999,999,999 target	Amount formatted correctly; layout intact	"$800.00 saved of $999,999,999.00"; layout intact	Pass
B11	Calendar: add a purchase whose description is only spaces	Rejected; asks for a description	Dialog stayed open asking for a description; nothing added	Pass
B12	Calendar: add a purchase of $0	Blocked; nothing added	Blocked by the form; nothing added	Pass
B13	Calendar: add a purchase of $10.005	Blocked; nothing added	Blocked by the form; nothing added	Pass
B14	Calendar: log a $1,000,000 purchase	Totals update; shows spending is over income; nothing breaks	Income left -$998,490.00; tip says purchases exceed income; chart and layout intact	Pass
B15	Calendar: name a purchase `<script>alert(1)</script>`	Shown as plain text; no code runs	Shown as plain text; no script ran	Pass
B16	Calendar: add a purchase dated a year from now	Shown in that month; this month's spending unchanged	Calendar jumped to October 2027; this month still $1,490.00	Pass

ID	Test	Expected	Actual	Pass / Fail
B17	Accounts: transfer from Checking to Checking	Rejected with a message	Error: "Choose two different accounts."	Pass
B18	Accounts: simulate a $5,000 transfer from Checking ($2,200 balance)	Rejected; balances unchanged	Error: "Not enough demo money in the source account."; Checking still $2,200.00	Pass
B19	Accounts: pay $200 to a credit card that owes $150	Rejected; amount owed never goes below $0	Error: "This payment is more than the credit card amount owed."; card still owes $150.00	Pass
B20	Accounts: save a transfer rule for $0	Blocked; no rule saved	Blocked (minimum $0.01); no rule saved	Pass
B21	Accounts: monthly transfer on Jan 31, simulate it twice	Next dates Feb 28, then back to Mar 31	Next dates Feb 28, then Mar 28: the rule stays on the 28th from then on	Fail
B22	Accounts: simulate a $1,000 transfer from Checking 4 times in a row	Stops when money runs out; never negative	Stopped at $200.00 with "Not enough demo money in the source account."	Pass
B23	Accounts: connect a demo account whose bank name is only spaces	Rejected; asks for a name	Dialog stayed open asking for a bank name	Pass
B24	Accounts: connect a demo account with a -$500 starting balance	Blocked; nothing added	Blocked (minimum $0); nothing added	Pass
B25	Accounts (phone width): connect an account with a 60-letter name and no spaces	Name wraps; page does not scroll sideways	Name wraps inside the card; no sideways scroll	Pass

ID	Test	Expected	Actual	Pass / Fail
B26	Pie Chart: open with a bad category in the link (`<script>` , `constructor` , `__proto__`)	Ignores it and shows Home	Fell back to Home; no errors	Pass
B27	All pages: corrupt the saved data in browser storage, then reload	Loads sample data instead of crashing	Loaded the sample data (spent $1,490.00)	Pass
B28	All pages: block browser storage (private mode) and add a purchase	App still works and warns changes will not be kept	Works; status: "Added to the calendar. Browser storage is unavailable; changes last until you close this page."	Pass
B29	Two tabs: add a purchase in tab A, then save allocations in tab B that was opened earlier	Both changes kept	Purchase from tab A was erased when tab B saved	Fail
B30	Any page: press Reset demo, choose Cancel, then press it again and choose OK	Cancel changes nothing; OK restores the sample	Cancel kept the data; OK restored the sample	Pass
B31	All pages at 375 px phone width	No page scrolls sideways	No page scrolls sideways	Pass
B32	Pie Chart: keyboard only, focus the Food slice and press Enter	Food details open	Selected Food with Enter	Pass
B33	Transactions: fix a purchase logged with the wrong amount ($180 Groceries to $18.50)	Purchase can be edited and totals update	Groceries changed to $18.50; spent this month now $1,328.50	Pass

ID	Test	Expected	Actual	Pass / Fail
B34	Goals: delete a goal added by mistake	Goal is deleted after confirming; balances unchanged	Goal removed ("No savings goals yet"); savings still $800.00	Pass
B36	Goals: delete the only goal, then reload Goals and Home	Stays empty; the sample goal does not come back	After reload Goals shows "No savings goals yet", Home shows "No savings goals yet"	Pass
B37	Accounts: remove a demo account added by mistake	Account can be deleted	No way to delete a demo account	Fail
B38	Transactions: edit a purchase to a blank description, then to $0	Both blocked; nothing changed	Both blocked; dialog stayed open and nothing changed	Pass
B39	Transactions: edit a purchase to $2,000,000,000	Blocked by the maximum amount	Blocked (maximum $1,000,000,000)	Pass
B40	Transactions: delete the $65 Gas purchase, first Cancel then OK	Cancel keeps it; OK removes it and totals drop by $65	Cancel kept it; OK deleted it; spent now $1,425.00	Pass
B41	Ask Budgetron: send a question with the app opened as a file (no server running)	Explains how to start the server; nothing breaks	Shows: "AI chat needs the local server. Run Start Budgetron.bat, then use the website it opens." with a link	Pass
B42	Ask Budgetron: paste a 3,000-character message	Cut to the 2,000-character limit	Message box kept 2,000 characters	Pass
B43	Ask Budgetron: drag the chat button far off the bottom-right of the screen	Button stays fully on screen	Button stayed fully on screen	Pass

ID	Test	Expected	Actual	Pass / Fail
B35	All tests above: watch the browser console for script errors	No errors	No errors	Pass

AI chat server tests

ID	Test	Expected	Actual	Pass / Fail
S1	Server: request the private .env file (where the API key lives)	Blocked	HTTP 404	Pass
S2	Server: request the server source code	Blocked	HTTP 404	Pass
S3	Server: path trick /pages/../.env	Blocked	HTTP 404	Pass
S4	Server: request with a non-local Host header (DNS rebinding)	Refused (403)	HTTP 403	Pass
S5	Chat: POST from another website (cross-site Origin)	Refused (403)	HTTP 403: "Send chat requests from the Budgetron website."	Pass
S6	Chat: send plain text instead of JSON	Refused (415)	HTTP 415	Pass
S7	Chat: send a 4,001-character message	Refused (400)	HTTP 400: "Send a question of up to 4,000 characters."	Pass
S8	Chat: send an 18-message history	Refused (400)	HTTP 400	Pass
S9	Chat: inject a fake "system" message	Refused (400)	HTTP 400	Pass

ID	Test	Expected	Actual	Pass / Fail
S10	Chat: attach a 17,000-character budget snapshot	Refused (400)	HTTP 400: "The budget snapshot is too large. Turn off budget sharing and try again."	Pass
S11	Chat: ask a question with no Gemini key set	Clear message to add the key	HTTP 503: "Add your Gemini API key to .env and save the file. Then send your question again."	Pass
S12	Chat: GET instead of POST	Refused (405)	HTTP 405	Pass
S13	Chat: send 11 questions within one minute	The 11th is refused with "wait a minute" (429)	First 10 accepted (they failed later only because a fake key was used); 11th refused with HTTP 429	Pass